

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA0922

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 6

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I A. Shane Rooney (192511057) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A. Shane Rooney

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas

dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface `MedicalRecord` containing methods `addRecord()` and `displayRecord()`. Implement the interface in classes `Patient` and `Doctor`. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

8. Online Shopping Order System using Packages

Create two packages:

- `shopping.product`
- `shopping.order`

Implement classes `Product` and `Order` in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

9. Student Result Analyzer using Arrays of Objects

Design a `Student` class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class `Device` and derived classes `Light`, `Fan`, and `AirConditioner`. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Answer :

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class `Shape` with an abstract method `area()`. Implement subclasses `Circle`, `Rectangle`, and `Triangle`. Use an array of `Shape` references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

SHAPE AREA CALCULATOR USING ABSTRACT CLASS & POLYMORPHISM

Aim

To design and implement an abstract class Shape with an abstract method area() and calculate the area of different shapes using subclasses and runtime polymorphism.

Objective

The objective of this experiment is to understand the concepts of Abstract Class and Polymorphism in Java. This program demonstrates how different objects can share a common interface while providing their own implementation for calculating the area of various geometric shapes.

Theory

Object-Oriented Programming (OOP) is a programming paradigm that focuses on creating programs using classes and objects. Two important concepts of OOP are Abstraction and Polymorphism.

Abstraction

Is the process of hiding the implementation details and exposing only the essential functionality. In Java, abstraction is achieved using abstract classes and interfaces. An abstract class cannot be instantiated directly and may contain both abstract and non-abstract methods. An abstract method is declared without a method body and must be implemented by all subclasses.

Polymorphism

means "one interface, many forms." Runtime polymorphism is achieved through method overriding, where subclasses provide their own implementation of a method declared in the parent class. The method that gets executed is determined at runtime based on the type of object.

In this experiment, an abstract class named Shape is created with an abstract method called area(). Three subclasses—Circle, Rectangle, and Triangle—inherit from the Shape class and implement the area() method using their respective mathematical formulas.

A Shape array is used to store objects of different subclasses. When the area() method is called through the Shape reference, Java automatically invokes the appropriate overridden method for each object. This demonstrates runtime polymorphism and makes the program flexible, reusable, and easy to extend.

Software Requirements

- Operating System: Windows 10/11
- Programming Language: Java
- JDK: Version 8 or above
- IDE: Eclipse / IntelliJ IDEA / VS Code / NetBeans

Algorithm

1. Create an abstract class named Shape.
2. Declare an abstract method area().
3. Create subclasses Circle, Rectangle, and Triangle.
4. Override the area() method in each subclass.
5. Create an array of Shape references.
6. Store objects of different subclasses in the array.
7. Invoke the area() method for each object.
8. Display the calculated area of all shapes.

Test Cases

Test Case	Input	Expected Output
TC1	Circle (Radius = 7)	Area = 153.94
TC2	Rectangle (Length = 10, Width = 5)	Area = 50.00
TC3	Triangle (Base = 8, Height = 4)	Area = 16.00
TC4	Objects stored in Shape array	Correct area displayed for all shapes using polymorphism

Expected Output

Circle Area = 153.94

Rectangle Area = 50.00

Triangle Area = 16.00

Advantages

- Demonstrates abstraction and runtime polymorphism.
- Improves code reusability.
- Reduces code duplication.
- Makes the program easy to maintain and extend.
- Follows Object-Oriented Programming principles.

Applications

- Computer graphics software.
- Engineering and CAD applications.
- Geometry calculation systems.

- Educational software.
- Scientific and mathematical applications.

Result

The program was executed successfully. The areas of Circle, Rectangle, and Triangle were calculated correctly using an abstract class and runtime polymorphism. The experiment demonstrates the effective implementation of abstraction, inheritance, method overriding, and polymorphism in Java.

Conclusion

This experiment provides a clear understanding of how abstract classes and runtime polymorphism work in Java. By using a common abstract class and overriding the area() method in different subclasses, the program becomes flexible, reusable, and easier to maintain. The use of an array of Shape references successfully demonstrates dynamic method invocation and object-oriented programming principles.